

## Development Environments

Development

Staging

Production

- ASPNETCORE\_ENVIRONMENT variable sets the **Runtime Environment**
- On development machine we set it in **launchsettings.json** file
- On Staging or Production server we set in the **operating system**
- Use **IHostingEnvironment** service to access the runtime environment
- Runtime environment defaults to **Production** if not set explicitly
- In addition to **standard environments** (Development, Staging, Production), **custom environments** (UAT, QA etc) are also supported

**Development Environment** : обичайно за софтиерните разработчици + JavaScript, CSS

**Staging Environment** : възможно най-идентичен с Production, като се цели безопасност и производителност, особено при B2B (Business to Business). Обикновено не се дебъгва и ремонтира, също се използват минимално JS, CSS, за да не товарят производителността. Ако има непредвидена грешка, ще се покаже само генерално съобщение от user friendly page, вместо от DeveloperExceptionPage.

**Production Environment** : Този софтуер, който използват потребителите ежедневно с максимална производителност и сигурност, заради което минимизираме JS, CSS, а грешките дават минимално количество информация.

\* Имаме и environment tag helpers in Razor View in HTML code.

"ASPNETCORE\_ENVIRONMENT": variable sets the Runtime Environment.

```
public void Configure(IApplicationBuilder app, IWebHostEnvironment env)
{
    if (env.IsDevelopment())
    {
        app.UseDeveloperExceptionPage();
    }

    app.UseStaticFiles();
    app.UseRouting();

    app.UseEndpoints(endpoints =>
    {
        endpoints.MapGet("/", async context =>
        {
            await context.Response.WriteAsync("Hosting environment: " + env.EnvironmentName);
        });
    });
}
```

Махаме новия код с грешките от предходния урок, като оставяме `app.UseStaticFiles();`

```
await context.Response.WriteAsync("Hosting environment: " + env.EnvironmentName);
```

за да изпише името на средата.



Hosting environment: Development

Съответно изписва Staging или Production, когато се сменят в launchSettings.json

Когато нямаме Environment, показва системната по подразбиране, която в Windows е Production, за по-голяма сигурност и производителност като потребителски достъп.

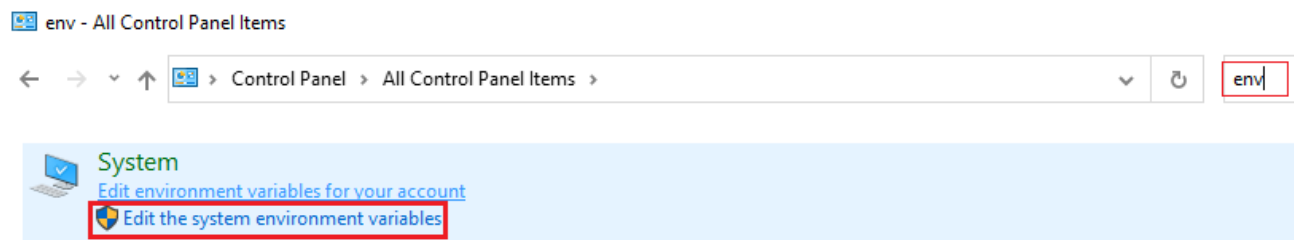


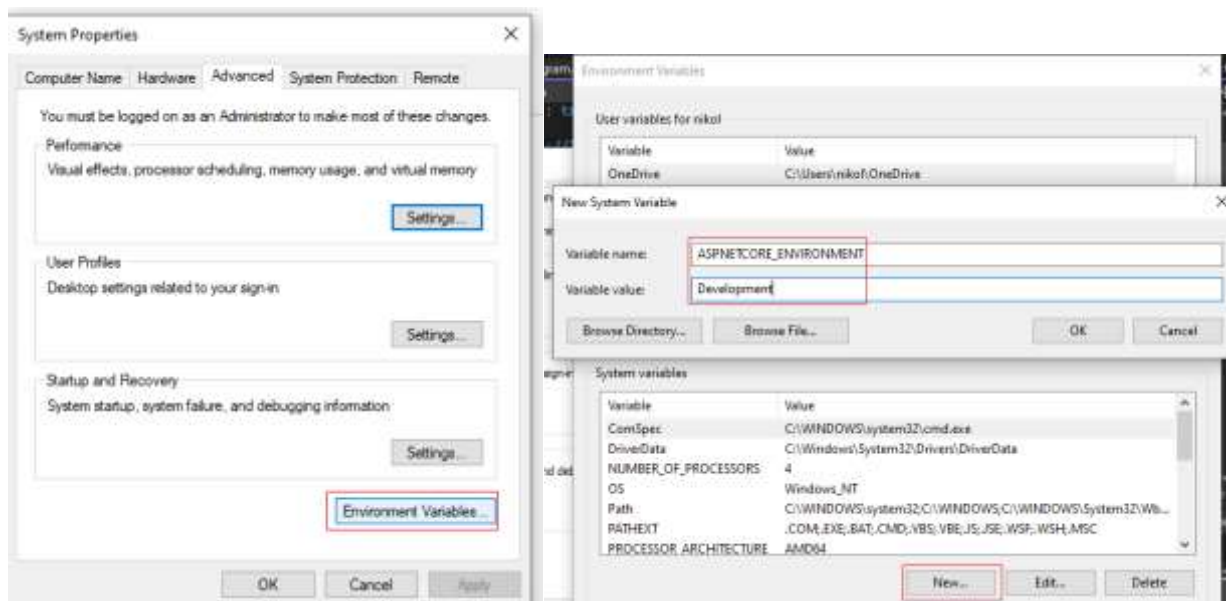
Hosting environment: Production

Properties -> launchSettings.json -> избираме Development, Staging, Production.

Търсачка 'control' -> C panel -> Още едно търсене: 'env' -> System -> Edit the system environment variables -> System properties -> Environment variables -> New -> ASPNETCORE\_ENVIRONMENT

По подразбиране в Windows = Production.





За да се отразят промените в брауъра трябва да се рестартира VS, за да може той да прихване поне веднъж промените от системата. Ако и това не помогне, след 1-я рестарт на компютъра, ще се отразят системните редакции.

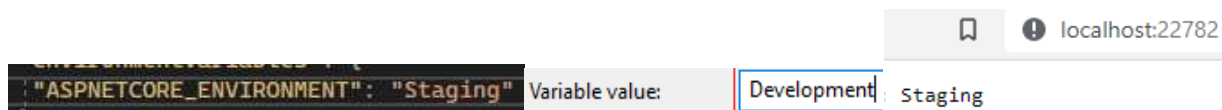
```

"profiles": {
  "IIS Express": {
    "commandName": "IISExpress",
    "launchBrowser": true,
    "environmentVariables": {
      // "ASPNETCORE_ENVIRONMENT": "Production"
    }
  }
}

```

Hosting environment: Development

Когато имаме в програмата VS Staging, а в OC Development, програмата е с приоритет и закрива системата.



Последно, в Startup.cs, имаме if с резултати в стойности 1 за истина и 0 за грешка.

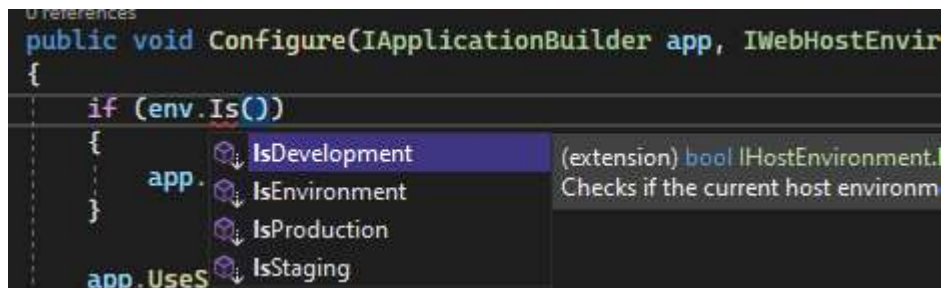
```

public void Configure(IApplicationBuilder
{
    if (env.IsDevelopment())
    {
        app.UseDeveloperPages();
    }
}

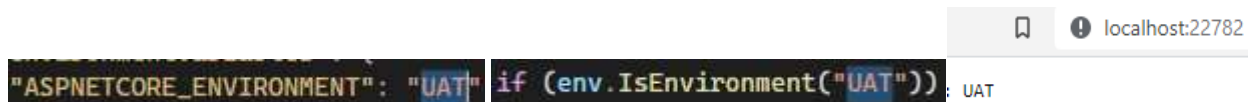
```

(extension) bool | Host Checks if the current host

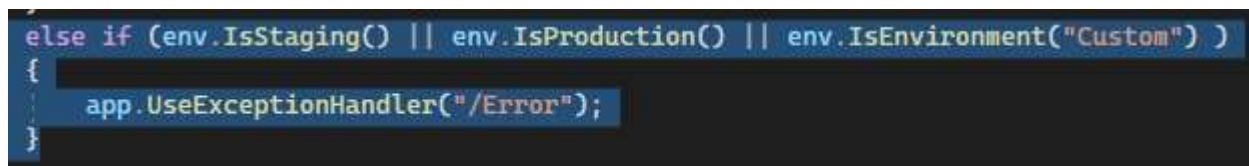
Освен IsDevelopment имаме избор IsProduction, IsStaging.



4-тият избор IsEnvironment се използва за custom персонализиран с име по наш избор:



Друг блок, който при мен в Windows 7 не се изпълни, но в 10 stana:



```
if (env.IsDevelopment()) // bool, IsStaging, IsProduction
{
    app.UseDeveloperExceptionPage();
}
```

Възможно е потребителска среда IsEnvironment("UAT")

```
if (env.IsDevelopment()) // bool, IsStaging, IsProduction, IsEnvironment("UAT")
{
    app.UseDeveloperExceptionPage();
}
else if (env.IsStaging() || env.IsProduction ||
env.IsDevelopment("UAT"))
{
    app.UseExceptionHandler("/Error.");
}
```

Локалните настройки засенчват глобалните